

Pooling, Normalisation & Activation Functions

Michael Vella

May 2026

1 Pooling

Pooling is a type of operation added to Convolutional Neural Networks (CNNs) following individual convolutional layers. It is used to reduce the dimensionality of images by reducing the number of pixels in the output from the previous convolutional layer while retaining the most important information from the previous layer. It is a deterministic downsampling operation, meaning it has no learnable parameters (no weights or biases to update via gradient descent) but is rather defined by several hyperparameters.

1.1 Hyperparameters

Pooling contains the following key hyperparameters:

1. **Pooling type**: Defines how the pooling aggregation is computed. The most common pooling types are max and average pooling.
2. **Filter Size (f)**: Defines the dimensions of the window (usually a 2 x 2 or 3 x 3 square) that slides across a feature map to reduce its spatial size.
3. **Stride (s)**: Determines how many pixels the window shifts at each step.
4. **Padding (p)**: Not frequently used but adds border pixels (usually zero) to the input to control the spatial output size.

1.2 Mathematics

For a feature map with dimensions $n_h \times n_w \times n_c$, the dimensions of the output after a pooling layer are:

$$\left\lfloor \frac{n_h - f}{s} \right\rfloor + 1 \times \left\lfloor \frac{n_w - f}{s} \right\rfloor + 1 \times n_c$$

Where:

- n_h : input height

- n_w : input width
- n_c : number of channels
- f : filter size
- s : stride length

Note: *Number of channels remains unchanged.*

1.3 Different Pooling Types

There are others but the main three types of pooling are:

1. **Max Pooling:** Selects the maximum element from the region of the feature map covered by the filter. The output after max pooling is performed would be a feature map preserving the most prominent features obtained from the previous layer. This provides very good performance in most cases.
2. **Average Pooling:** Instead of selecting the maximum element from each region, this pooling types averages all elements present in the region. This gives us the average of the features present in a particular pooling dimension. This provides a more generalised representation of the input so it is useful where preserving the overall context is important.
3. **Global Pooling:** Reduces each channel in the feature map to a single value, producing a $1 \times 1 \times n_c$ output. This is equivalent to applying a filter size of $n_h \times n_w$. Two types of global pooling which are global max pooling and global average pooling. Global max pooling takes the maximum across the entire feature map. Global average pooling averages all values in the feature map. Global pooling is often used at the end of a CNN to reduce feature maps to a single value per channel, effectively summarising spatial information.

1.4 Examples

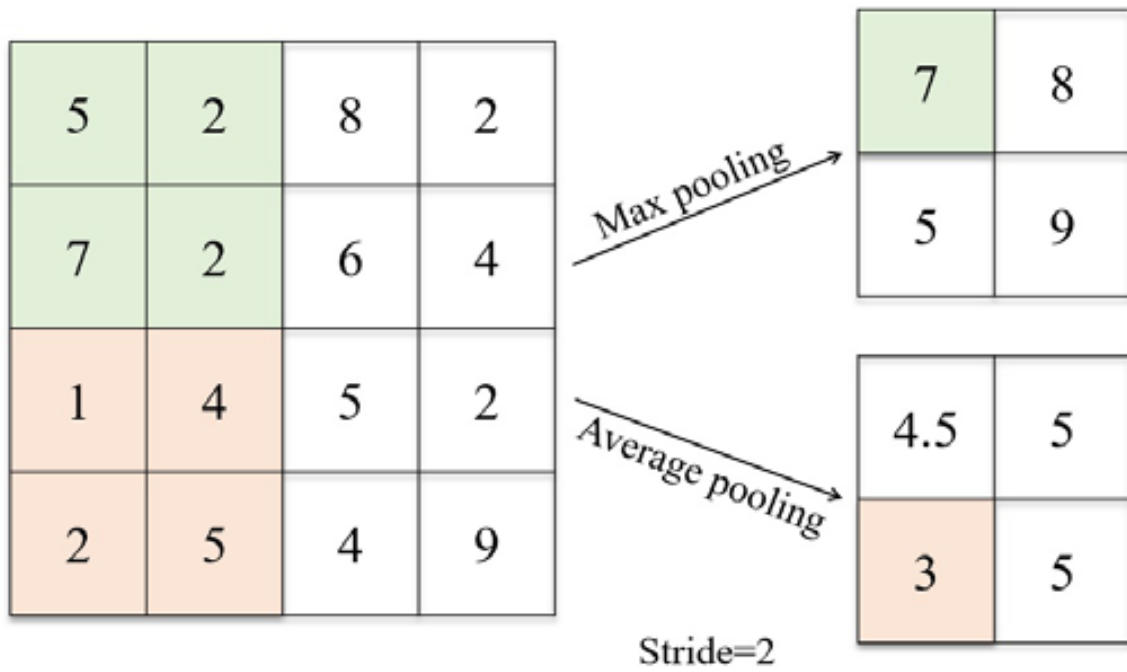


Figure 1: Example of max and average pooling using a 2×2 filter and a stride of 2.

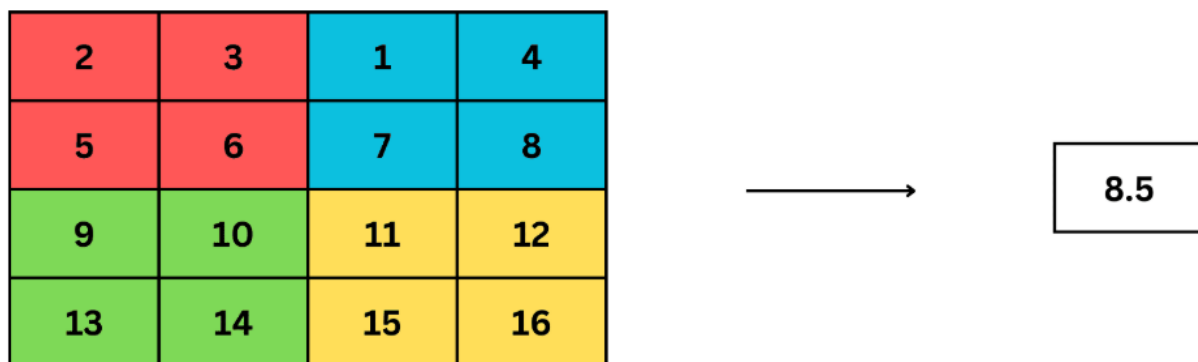


Figure 2: Example of global average pooling.

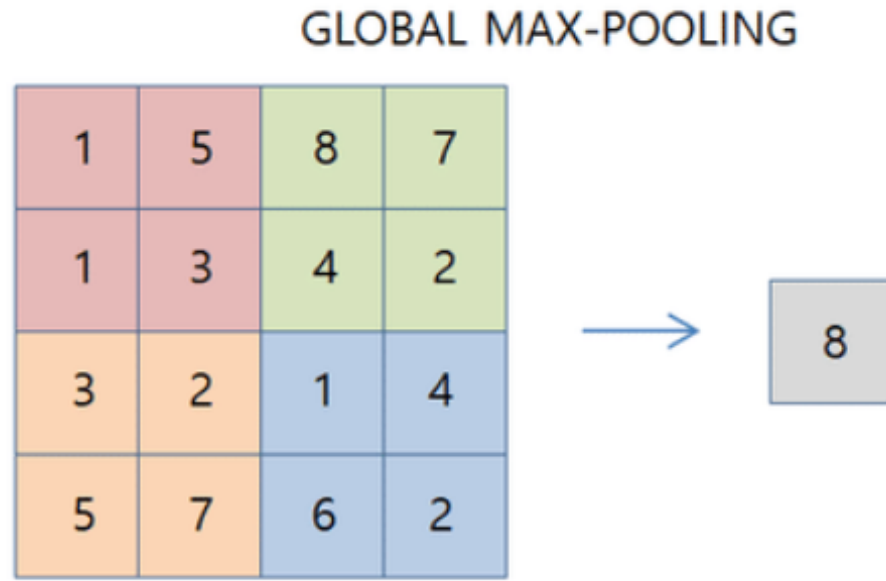


Figure 3: Example of global max pooling.

1.5 Uses & Limitations

Main advantages and uses of pooling are:

1. **Dimensionality Reduction & Efficiency:** Significantly reduces the spatial size of feature maps, which accelerates training as we lower the number of parameters and computations required.
2. **Translation Invariance:** Model is robust to small shifts, distortions, or translations in the input image.
3. **Controls Overfitting:** By summarising features and reducing the data complexity, it acts as a form of regularisation, helping to prevent the model from overfitting to training data.
4. **Feature Selection:** Retains important features (max pooling) or captures overall context (average pooling).

Main limitations of pooling are:

1. **Information Loss:** Since pooling is a form of data compression, pooling discards certain information, some of which may be important.
2. **Hyperparameter sensitivity:** Choice of hyperparameters is important to avoid excessive loss of information.
3. **Over-smoothing:** Improper design or excessive pooling can blur crucial features.

Examples of real-world applications that use pooling:

1. **Image Classification (e.g. VGG, AlexNet):** Pooling layers are stacked after convolutional layers throughout the network. These are used to progressively shrink the spatial dimensions so that when data reaches the fully connected dense layers, the number of parameters is manageable without sacrificing the learned features.
2. **Face Recognition:** Systems like FaceNet use global average pooling before the embedding layer to produce a compact, fixed-size descriptor of a face regardless of input resolution. This makes the system robust to minor shifts or crops in the input image.

1.6 Quick Reference

1.6.1 Key Equations

Description	Formula
Output Height	$\left\lfloor \frac{n_h - f}{s} \right\rfloor + 1$
Output Width	$\left\lfloor \frac{n_w - f}{s} \right\rfloor + 1$
Channels (unchanged)	n_c
Global Pooling Filter Size	$f = n_h \times n_w$

Table 1: Key pooling equations.

1.6.2 Hyperparameter Typical Ranges

Hyperparameter	Typical Values
Filter size (f)	2×2 (most common), 3×3
Stride (s)	2 (most common), 1
Padding (p)	0 (rarely used in pooling)

Table 2: Hyperparameters used in pooling.

1.6.3 Pooling Types at a Glance

Type	Operation	Best Used When
Max Pooling	Takes maximum in window	Preserving prominent features
Average Pooling	Averages all values in window	Preserving overall context
Global Max Pooling	Max across entire feature map	Feature presence detection
Global Average Pooling	Mean across entire feature map	End-of-network summarisation

Table 3: Summary of pooling types and when best to use which.

1.6.4 Key Terminology

Translation Invariance The model produces the same or very similar output even if the input is slightly shifted. Pooling contributes to this but does not make a network fully invariant on its own.

Downsampling Reducing the spatial dimensions of a feature map. Pooling is one way to achieve this.

Feature Map The output of a convolutional layer which is a 3D volume of shape $n_h \times n_w \times n_c$ representing learned spatial features.

1.6.5 Common Misconceptions

- **Pooling has learnable parameters.** It does not. Pooling is entirely defined by its hyperparameters and no weights are updated during back propagation.
- **Pooling always improves performance.** Excessive or poorly configured pooling discards too much spatial information, which can harm tasks requiring precise localisation such as semantic segmentation.
- **Max pooling is always better than average pooling.** Max pooling tends to outperform when detecting whether a feature exists matters most. Average pooling can outperform when the overall distribution of activations carries meaning, particularly in global summarisation contexts.

1.7 Key Papers

1. *Evaluation of Pooling Operations in Convolutional Architectures for Object Recognition* [1].

URL: https://link.springer.com/chapter/10.1007/978-3-642-15825-4_10

Summary: This paper provides one of the first direct empirical comparisons of pooling operations within a fixed CNN architecture, evaluated on the Caltech-101 and NORB datasets. The authors compare max pooling against subsampling (similar to average pooling but is followed by multiplying by a trainable parameter and adding a trainable bias), finding that max pooling significantly outperforms subsampling in both convergence speed and recognition accuracy. They also investigate overlapping pooling windows and various window functions, concluding that neither offers consistent improvement over standard non-overlapping max pooling.

Relevance: This paper is directly relevant as it provides controlled experimental evidence for why max pooling (and pooling in general) became the dominant choice in CNN architectures.

2. *Network In Network* [2].

URL: <https://arxiv.org/abs/1312.4400>

Summary: This paper introduces the Network In Network (NIN) architecture, which replaces standard linear convolutional filters with small multilayer perceptrons to better model non-linear relationships within local receptive fields. Its most significant contribution to pooling is the proposal of global average pooling (GAP) as a direct replacement for fully connected layers at the end of a CNN. GAP reduces each feature map to a single value per channel, acting as a structural regulariser that enforces correspondence between feature maps and output categories, while substantially reducing parameters and overfitting.

Relevance: This paper is the original source proposing global average pooling, making it a foundational reference for this topic. Its influence is far-reaching, as GAP was subsequently adopted by major architectures including GoogLeNet and MobileNet, cementing it as a standard design choice in modern CNNs.

1.8 Additional (Informal) Material

1. [\[Medium\] Pooling and their types in CNN](#)
2. [\[GeeksforGeeks\] Introduction to Pooling Layer in CNN](#)
3. [\[Article\] What are Max Pooling, Average Pooling, Global Max Pooling and Global Average Pooling?](#)
4. [\[YouTube\] Max Pooling in Convolutional Neural Networks explained](#)

2 Batch Normalisation

Normalisation is a pre-processing technique used in machine learning to scale numerical data to a standard consistent range (for example, a mean of 0 and a variance of 1). The idea behind it is to prevent features with larger scales from overpowering those features with smaller scales, allowing for more network stability and accuracy.

Usually, normalisation is done before we start training (on the dataset as a whole). Then, this normalised dataset is passed as input to the network. However, in very deep neural networks, as this data gets passed on down the layers, this data might once again become denormalised. This is where batch normalisation comes in.

Batch normalisation (BatchNorm) is the process of running this normalisation step (scaling to a mean of 0 and a variance of 1) at every layer of the network. Essentially, for every mini-batch of data, we apply this batch normalisation step and scale the inputs. Since during training we don't process the entire dataset at once (as it would be too slow) and split the dataset into mini-batches, BatchNorm computes the **mean** (μ_B) and **variance** (σ_B^2) for each mini-batch separately and uses these for normalisation.

When originally proposed (and in most cases), this batch normalisation step is applied before the activation function (*Conv* → *BatchNorm* → *Activation*) to keep inputs to the activation

function within a stable range. However, there are certain implementations that apply BatchNorm after the activation function (*Conv* $\rightarrow$ *Activation* $\rightarrow$ *BatchNorm*) and achieve good results as well.

2.1 Parameters

A BatchNorm block contains the following parameters:

1. **Beta** (β): Used as an offset parameter to undo the normalisation if needed. This is a learned parameter (learned during training via back propagation). Initially initialised as 0.
2. **Gamma** (γ): Acts as a multiplier on normalised inputs, allowing the network to amplify or suppress features. This is a learned parameter (learned during training via back propagation). Initially initialised as 1.
3. **Moving Mean** (μ): Tracks the mean of all batches. Not a learnable parameter but a statistical parameter updated after each mini-batch iteration.
4. **Moving Variance** (σ^2): Tracks the variance of all batches. Not a learnable parameter but a statistical parameter updated after each mini-batch iteration.

2.2 Mathematics

The mathematics for BatchNorm can be seen in Figure 4

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;	
Parameters to be learned: γ, β	
Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$	
$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$	// mini-batch mean
$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$	// mini-batch variance
$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$	// normalize
$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i)$	// scale and shift

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Figure 4: Reproduced from [3].

Equations for calculating the moving mean and moving variance are:

- $\hat{\mu}_{new} = \alpha\hat{\mu}_{old} + (1 - \alpha)\mu_B$ where α is a hyperparameter usually set to 0.9 or 0.99. Typically, for the first mini-batch, the moving mean would be the mean of the first batch.
- $\hat{\sigma}_{new}^2 = \alpha\hat{\sigma}_{old}^2 + (1 - \alpha)\sigma_B^2$ where α is a hyperparameter usually set to 0.9 or 0.99. Typically, for the first mini-batch, the moving variance would be the variance of the first batch.

2.2.1 Working Example

Assume a dataset of four entries with values $[1, 5, 15, 30]$ and assume $\gamma = 1$, $\beta = 0$ and $\alpha = 0.99$. This dataset is split into two batches. Batch one would have $[1, 5]$ and batch two would have $[15, 30]$.

Batch one vector is $[1, 5]$:

- **Batch Mean (μ_1):** $1/2(1 + 5) = 3$
- **Batch Variance (σ_1^2):** $1/2((1 - 3)^2 + (5 - 3)^2) = 4$
- **Moving Mean (μ):** 3.
- **Moving Variance (σ^2):** 4.
- $\hat{x}^1$ **Result:** $(1 - 3)/\sqrt{4 + 0.000001} \approx -1$
- $\hat{x}^2$ **Result:** $(5 - 3)/\sqrt{4 + 0.000001} \approx 1$
- **Output vector:** $[-1, 1]$

Batch two vector is $[15, 30]$:

- **Batch Mean (μ_2):** $1/2(15 + 30) = 22.5$
- **Batch Variance (σ_2^2):** $1/2((15 - 22.5)^2 + (30 - 22.5)^2) = 56.25$
- **Moving Mean (μ):** $0.99 \times 3 + 0.01 \times 22.5 = 3.195$
- **Moving Variance (σ^2):** $0.99 \times 4 + 0.01 \times 56.25 = 4.5225$
- $\hat{x}^1$ **Result:** $(15 - 22.5)/\sqrt{56.25 + 0.000001} \approx -1$
- $\hat{x}^2$ **Result:** $(30 - 22.5)/\sqrt{56.25 + 0.000001} \approx 1$
- **Output vector:** $[-1, 1]$

NOTE: Important to note that these batch mean/variance calculations are only calculated during training. Since during inference we no longer have batches but perform inference one at a time, we can no longer compute these. As a replacement, we then use the moving mean and the moving variance. In fact, these moving values are never used in training but are only used during inference.

2.3 Uses & Limitations

Main advantages and uses of BatchNorm are:

1. **Faster Convergence:** In traditional networks as the input data propagates through the network, the distribution of each layer's inputs changes. This phenomenon is known as internal covariate shift and it can slow down the training process. BatchNorm reduces this, which allows for faster convergence during training.
2. **High Learning Rates:** Since data is going to be normalised, slightly higher learning rates can be used without running the risk of divergence.
3. **Regularisation effect:** Intrinsically introduces a slight regularisation effect that reduces the need for adding additional regularisation techniques like dropout.
4. **Helps avoid vanishing/exploding gradients:** Data would be normalised and scaled down which handles the vanishing/exploding gradients problem.

Main limitations are:

1. **Computational Overhead:** Although the technique improves convergence speed, it adds overhead because it requires additional calculations (calculating mean and variance for every mini-batch).
2. **Dependency on batch size:** Requires a relatively large batch size to accurately calculate mean and variance. If batch size is too small, it leads to noisy statistics. BatchNorm cannot be applied to networks which use a batch-size of 1.
3. **Discrepancy between training and inference:** During training, normalisation is performed using mini-batch statistics but at inference, we can only use the moving average. If the data distribution changes, this inconsistency can lead to performance degradation.

2.4 Quick Reference

2.4.1 Key Equations

- **Batch Mean:** $\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$
- **Batch Variance:** $\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$
- **Normalisation:** $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$
- **Scale and Shift:** $y_i = \gamma \hat{x}_i + \beta$
- **Moving Mean Update:** $\hat{\mu}_{new} = \alpha \hat{\mu}_{old} + (1 - \alpha) \mu_B$
- **Moving Variance Update:** $\hat{\sigma}_{new}^2 = \alpha \hat{\sigma}_{old}^2 + (1 - \alpha) \sigma_B^2$

2.4.2 Parameters and Typical Values

Parameter	Type	Default / Range
γ (scale)	Learned	Initialised to 1
β (shift)	Learned	Initialised to 0
ϵ	Hyperparameter	10^{-5} to 10^{-3}
α (momentum)	Hyperparameter	0.9 – 0.99
Batch size	Hyperparameter	≥ 16 recommended

2.4.3 Placement in Network

- **Standard (original paper):** Conv $\rightarrow$ BatchNorm $\rightarrow$ Activation
- **Alternative:** Conv $\rightarrow$ Activation $\rightarrow$ BatchNorm

2.4.4 Training vs Inference

	Training	Inference
Mean used	Mini-batch mean μ_B	Moving mean $\hat{\mu}$
Variance used	Mini-batch variance σ_B^2	Moving variance $\hat{\sigma}^2$

2.4.5 Key Terminology

- **Internal Covariate Shift:** The change in distribution of layer inputs during training as the preceding layer’s weights update. BatchNorm reduces this.

2.4.6 Common Misconceptions

- **Moving statistics are used during training:** The moving mean and variance are *only* used at inference. During training, the per-batch statistics μ_B and σ_B^2 are always used for normalisation.
- **γ and β are fixed:** Both are *learned* parameters updated via back propagation, not hyperparameters you set manually.
- **BatchNorm always improves performance:** With very small batch sizes it can hurt performance due to noisy statistics.
- **ϵ is negligible:** While small, ϵ is essential for numerical stability; without it, division by zero occurs when variance is zero.
- **BatchNorm replaces all regularisation:** It introduces a *mild* regularisation effect but is not a substitute for dropout in tasks where strong regularisation is needed.

2.5 Key Papers

1. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift* [3].

URL: <https://arxiv.org/abs/1502.03167>

Summary: This paper introduces Batch Normalisation as a solution to internal covariate shift (the continuous change in layer input distributions during training). The authors propose normalising each mini-batch's activations and introducing two learned parameters, gamma and beta, to preserve the network's representational capacity. Evaluated on ImageNet using a variant of the Inception architecture, the method achieved the same accuracy as the baseline using 14 times fewer training steps, and surpassed state-of-the-art results. The paper also demonstrates that BatchNorm enables higher learning rates, reduces the need for Dropout, and allows training with saturating non-linearities such as sigmoid.

Relevance: This is the original paper proposing Batch Normalisation, making it the primary reference for this topic. All mathematical formulations, parameter definitions, and placement conventions discussed in these notes derive directly from this work.

2.6 Additional (Informal) Material

1. [\[GeeksforGeeks\] Internal Covariant Shift Problem in Deep Learning](#)
2. [\[GeeksforGeeks\] What is Batch Normalization In Deep Learning?](#)
3. [\[Medium\] Batch Normalization in Machine Learning](#)
4. [\[Article\] Batch Norm Explained Visually – How it works, and why neural networks need it](#)
5. [\[YouTube\] How Does Batch Normalization Work](#)

3 Activation Functions

An activation takes the values generated by a given network layer (in a fully connected network, this would be the sum of weights and biases) and apply a certain transformation to these values to map them to a specific range (see Figure 5). As the name suggests, it is some kind of function that should "activate" the neuron and it is mainly used to introduce non-linearity, which enables the model to learn and represent complex data patterns. Without it, even a deep neural network would behave like a simple linear regression model.

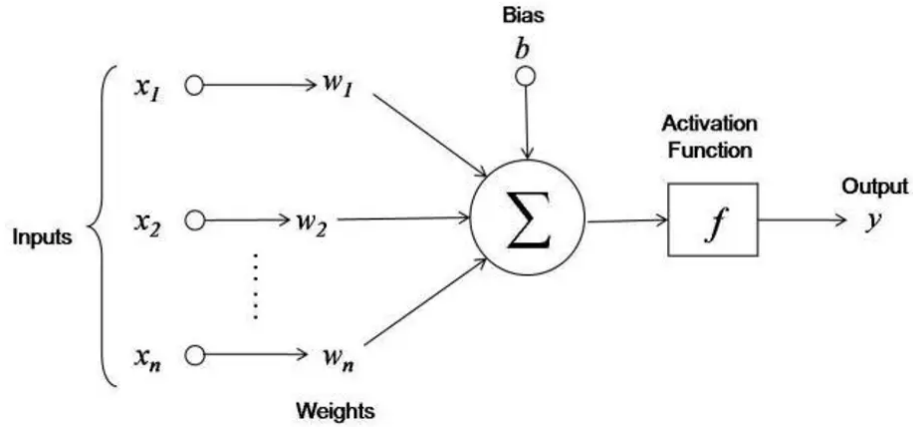


Figure 5: Activation function inside a neural network.

The majority of activation functions are non-linear. This is because non-linear functions help introduce additional complexity into neural networks to help facilitate the learning process. Linear functions are then usually used in the output layer of a regression model. For example, to predict continuous values, a linear activation function in the output layer makes sense because the predictions should not be bounded.

One important property of an activation function is that it should be differentiable. This is required to enable neural networks to train using gradient-based optimisation (back propagation). Differentiation allows the model to calculate the gradient of the loss function with respect to each weight, determining how to adjust parameters to minimise errors.

3.1 Different Types of Activation Functions

As seen in Figure 6, various activation functions exist.

Neural Network Activation Functions: a small subset!

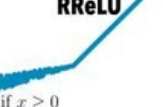
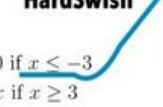
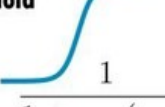
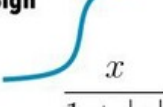
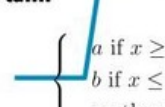
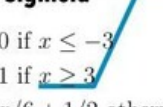
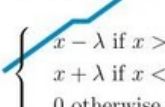
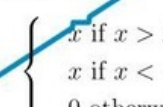
ReLU  $\max(0, x)$	GELU  $\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$	PReLU  $\max(0, x)$
ELU  $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish  $\frac{x}{1 + \exp -x}$	SELU  $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus  $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish  $x \tanh \left(\frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$	RReLU  $\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$
HardSwish  $\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid  $\frac{1}{1 + \exp(-x)}$	SoftSign  $\frac{x}{1 + x }$
Tanh  $\tanh(x)$	Hard tanh  $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid  $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink  $x - \tanh(x)$	Soft Shrink  $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink  $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

Figure 6: A small subset of the activation functions that exist.

However, the most common activation functions used in modern neural networks are those from the ReLU family, favoured for training deep networks efficiently.

3.1.1 ReLU

Widely used activation function that outputs the input directly if it is positive, and zero otherwise. Mathematically, the function is denoted as $f(x) = \max(0, x)$ and the output range is $[0, \infty)$.

Advantages include:

1. **Computational Efficiency:** Simple to calculate, requiring only a max operation. This makes training deep neural networks much faster when compared to other functions.
2. **Sparse Activation:** Outputting zero for negative inputs creates a sparse network, meaning that only a fraction of neurons are activated at a given time. This leads to more efficiency.

and faster computation.

3. **Gradient Computation:** Its derivative is simple. It's either 0 (when the input is negative) or 1 (when the input is positive). This helps to avoid the vanishing gradient problem (gradients to update weights during back propagation become extremely small, preventing learning), which is a common issue with other activation functions.

Limitations include:

1. **Dying ReLU Problem:** One of the most significant drawbacks of ReLU is the "dying ReLU" problem. When a neuron consistently receives negative pre-activations — often caused by a large learning rate or poor weight initialisation — its gradient becomes permanently zero. Since back propagation has nothing to work with, the weights never update, locking the neuron into always outputting zero regardless of the input. Once this happens, the neuron is effectively dead for the remainder of training.
2. **Unbounded Output:** Since the positive output range is ∞ , this can sometimes result in very large values, resulting in exploding gradients or training instability.

3.1.2 Leaky ReLU

A modified version of the ReLU activation function designed to tackle the "dying ReLU" problem. It introduces a slight modification to the ReLU activation function where instead of outputting zero for negative values, it output a small non-zero value.

Mathematically, the function is denoted as

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha \cdot x & \text{if } x \leq 0 \end{cases} = \max(\alpha x, x), \quad \text{where } \alpha \text{ is a small constant (e.g., } \alpha = 0.01)$$

Advantages include:

1. **Tackles the dying ReLU problem:** By allowing a small negative slope ($\alpha \cdot x$) neurons continue to learn even with negative input, preventing them from becoming permanently inactive.
2. **Low computational cost:** Similar to standard ReLU, it is highly efficient to calculate, making it faster than exponential-based functions.
3. **More consistent gradient flow:** Maintains gradient flow during back propagation, which could result in more robust learning when compared to standard ReLU.

Limitations include:

1. **Additional Hyperparameter:** Requires setting the negative slope parameter α , which might need tuning for optimal performance.
2. **Not Always Better:** Performance improvements over standard ReLU are not guaranteed and can be marginal on some datasets.
3. **Fixed Slope:** The slope α is typically fixed and constant, whereas other functions learn this parameter, potentially limiting adaptability.

3.1.3 Sigmoid

A smooth, probabilistic activation function that squashes any real-valued input into the range $(0, 1)$. Mathematically, the function is defined as

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

and its derivative, used during back propagation, can be expressed conveniently in terms of the output itself as $f'(x) = \sigma(x)(1 - \sigma(x))$. The S-shaped curve is symmetric around the point $(0, 0.5)$, and outputs can be interpreted naturally as probabilities.

Advantages include:

1. **Probabilistic Interpretation:** The output range of $(0, 1)$ maps directly to probabilities, making Sigmoid the natural choice for the output layer of binary classification networks. A neuron outputting 0.85 can be directly interpreted as an 85% confidence in the positive class.
2. **Smooth and Differentiable:** Sigmoid is continuously differentiable everywhere, providing smooth gradients throughout the entire input domain. This property was historically attractive for gradient-based optimisation and remains useful in gating mechanisms where smooth transitions between states are required.
3. **Bounded Output:** The strictly bounded output range prevents individual neuron activations from growing arbitrarily large, offering a degree of inherent stability that unbounded functions such as ReLU do not provide.

Limitations include:

1. **Vanishing Gradient Problem:** For inputs with large absolute values, the function saturates and its gradient approaches zero, since $f'(x) = \sigma(x)(1 - \sigma(x)) \rightarrow 0$ as $|x| \rightarrow \infty$. When gradients are multiplied across many layers during back propagation, they shrink exponentially, effectively preventing earlier layers from receiving a meaningful learning signal.
2. **Non-Zero-Centred Output:** Because all outputs are strictly positive, the gradients passed to weights in a given layer will always share the same sign. This forces weight updates to move together in the same direction, causing inefficient, zig-zagging optimisation dynamics rather than direct convergence.

3. **Computational Cost:** Evaluating Sigmoid requires computing an exponential at every neuron, which is considerably more expensive than the simple comparison operation used by ReLU. This cost accumulates significantly in deep networks with large numbers of neurons.

3.1.4 Tanh

A smooth, zero-centred activation function that squashes any real-valued input into the range $(-1, 1)$. Mathematically, the function is defined as

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

and is closely related to the sigmoid function. Unlike ReLU, Tanh is differentiable everywhere and its output is symmetric around zero.

Advantages include:

1. **Zero-Centred Output:** Because the output range is $(-1, 1)$, the mean activation is approximately zero. This is beneficial because it ensures that gradients during back propagation are both positive and negative, leading to more balanced and efficient weight updates compared to non-zero-centred functions such as sigmoid.
2. **Stronger Gradients than Sigmoid:** The gradient of Tanh reaches a maximum of 1 at $x = 0$, compared to a maximum of 0.25 for the sigmoid function. This means Tanh propagates stronger gradient signals during back propagation, making it a better choice in practice for hidden layers.
3. **Smooth and Differentiable:** Unlike ReLU, Tanh is continuously differentiable everywhere, which can be beneficial for optimisation landscapes that rely on smooth gradient information. Its derivative is given by $f'(x) = 1 - \tanh^2(x)$.

Limitations include:

1. **Vanishing Gradient Problem:** For inputs with large absolute values, the function saturates and its gradient approaches zero. During back propagation, this causes gradients to become extremely small as they are multiplied across layers, effectively preventing earlier layers from learning. This is the same fundamental problem that motivated the development of ReLU.
2. **Computational Cost:** Computing Tanh requires evaluating exponential functions, making it more computationally expensive than ReLU-based activations, which rely only on a simple comparison operation. This difference becomes significant when training very deep networks or working with large batch sizes.
3. **Not Sparse:** Unlike ReLU, Tanh never outputs exactly zero for a non-zero input, meaning all neurons remain active at all times. This prevents the sparse activation behaviour that contributes to ReLU's computational efficiency.

3.1.5 Softmax

A generalisation of the Sigmoid function to multiple classes, Softmax converts a vector of arbitrary real-valued scores (logits) into a probability distribution over K classes. Mathematically, for an input vector $\mathbf{z} = (z_1, z_2, \dots, z_K)$, the output for class i is defined as

$$f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, 2, \dots, K$$

Every output value lies in the range $(0, 1)$ and the outputs sum to exactly 1, forming a valid probability distribution. Softmax is almost exclusively used as the final activation in multi-class classification networks.

Advantages include:

1. **Valid Probability Distribution:** By normalising across all classes simultaneously, Softmax ensures outputs are mutually exclusive probabilities that sum to 1. This makes the network's output directly interpretable — an output of 0.75 for class i means the network assigns a 75% probability to that class given the input.
2. **Amplifies Relative Differences:** Because of the exponential in the numerator, Softmax disproportionately amplifies the largest logit relative to the others. This creates more confident, well-separated probability distributions, making the dominant class easier to identify during inference.
3. **Convenient Gradient with Cross-Entropy:** When paired with the cross-entropy loss, the gradient of the loss with respect to the input logits simplifies elegantly to $\hat{y}_i - y_i$, where $\hat{y}_i$ is the predicted probability and y_i is the ground truth. This clean gradient expression makes optimisation numerically stable and efficient to compute.

Limitations include:

1. **Numerical Instability:** Computing e^{z_i} for large values of z_i can cause floating-point overflow. In practice this is addressed by subtracting the maximum logit before exponentiation, yielding the numerically stable equivalent

$$f(z_i) = \frac{e^{z_i - \max(\mathbf{z})}}{\sum_{j=1}^K e^{z_j - \max(\mathbf{z})}}$$

which produces identical outputs but avoids overflow entirely.

2. **Mutual Exclusivity Assumption:** Softmax forces the probabilities of all classes to compete against one another. Increasing confidence in one class necessarily decreases confidence in all others. This makes it inappropriate for multi-label classification problems, where multiple classes can be simultaneously correct. In such settings, independent Sigmoid outputs per class are preferred.

3. **Overconfidence on Out-of-Distribution Inputs:** Softmax always produces a probability distribution regardless of input quality. Even for inputs that are entirely unlike anything seen during training, the network will still output high-confidence predictions for one class, since the outputs must sum to 1. This can make models appear more certain than they are, which is a significant concern in safety-critical applications.

3.1.6 GELU

The Gaussian Error Linear Unit (GELU) is a smooth, non-monotonic activation function that weights each input by the probability that it is greater than other inputs drawn from a standard normal distribution. Mathematically, the function is defined as

$$f(x) = x \cdot \Phi(x) = x \cdot P(X \leq x), \quad X \sim \mathcal{N}(0, 1)$$

where $\Phi(x)$ is the cumulative distribution function (CDF) of the standard normal distribution. Since $\Phi(x)$ has no closed form, the exact expression is computed using the error function erf :

$$f(x) = \frac{x}{2} \left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$$

Because this is computationally expensive, a fast approximation is widely used in practice:

$$f(x) \approx 0.5 \cdot x \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3)\right) \right)$$

The output range is approximately $[-0.17, \infty)$, and unlike ReLU, GELU is smooth and differentiable everywhere, including at the origin. It is the default activation function in prominent transformer-based architectures including BERT and GPT.

Advantages include:

1. **Smooth Non-Linearity:** GELU is continuously differentiable everywhere, with no sharp corners or discontinuities in its gradient. This produces smoother loss landscapes compared to ReLU, which can benefit optimisation in very deep networks and attention-based architectures where gradient flow across many layers is critical.
2. **Probabilistic Gating:** Rather than applying a hard threshold at zero as ReLU does, GELU softly gates each input based on its magnitude relative to the distribution of other inputs. Small negative values are attenuated but not entirely zeroed out, while large positive values pass through almost unchanged. This stochastic interpretation allows the network to retain weak signals that ReLU would discard entirely.
3. **Non-Monotonic Behaviour:** GELU is non-monotonic — it has a slight dip below zero before recovering. This introduces a small amount of curvature that is absent from ReLU

and Leaky ReLU, giving the network additional expressive power without requiring extra parameters.

Limitations include:

1. **Computational Cost:** Exact evaluation of GELU requires computing either the error function erf or the approximation involving a hyperbolic tangent and a cubic polynomial, both of which are considerably more expensive than the single comparison operation used by ReLU. While the approximation reduces this cost, it remains a measurable overhead in large-scale training.
2. **Not Sparse:** Like Tanh and Sigmoid, GELU never produces exactly zero output for non-zero inputs, forgoing the sparse activation behaviour that contributes to ReLU’s efficiency. Every neuron remains active to some degree at all times, increasing computational load relative to ReLU-based networks.
3. **Less Interpretable Behaviour:** The probabilistic gating mechanism, while theoretically motivated, makes GELU harder to reason about intuitively compared to ReLU. Debugging unexpected network behaviour or understanding why a particular neuron fires is less straightforward when the activation rule is a smooth statistical function rather than a simple threshold.

3.2 Quick Reference

3.2.1 Activation Function Summary

Function	Formula	Output Range	Typical Use Case
ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	Hidden layers (CNNs, MLPs)
Leaky ReLU	$f(x) = \max(\alpha x, x)$	$(-\infty, \infty)$	Hidden layers, dying ReLU fix
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	$(0, 1)$	Binary classification output
Tanh	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$(-1, 1)$	Hidden layers, RNNs
Softmax	$f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0, 1)$, sums to 1	Multi-class classification output
GELU	$f(x) = x \cdot \Phi(x)$	$[-0.17, \infty)$	Transformers (BERT, GPT)

Table 4: Summary of common activation functions.

3.2.2 Key Equations

- **ReLU derivative:** $f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$

- **Sigmoid derivative:** $f'(x) = \sigma(x)(1 - \sigma(x))$, maximum value 0.25 at $x = 0$
- **Tanh derivative:** $f'(x) = 1 - \tanh^2(x)$, maximum value 1 at $x = 0$
- **Numerically stable Softmax:** $f(z_i) = \frac{e^{z_i - \max(\mathbf{z})}}{\sum_j e^{z_j - \max(\mathbf{z})}}$
- **Softmax + Cross-Entropy gradient:** $\frac{\partial \mathcal{L}}{\partial z_i} = \hat{y}_i - y_i$
- **GELU (exact):** $f(x) = \frac{x}{2} \left[1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$
- **GELU (approximation):** $f(x) \approx 0.5x \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3)\right) \right)$

3.2.3 When to Use Each Function

- **ReLU:** Default choice for hidden layers in CNNs and MLPs as it is fast and effective in most settings.
- **Leaky ReLU:** Prefer over ReLU when dying neurons are observed during training.
- **Sigmoid:** Use only in the output layer of binary classification networks.
- **Tanh:** Preferred over Sigmoid for hidden layers when a zero-centred, bounded output is needed (e.g. RNNs).
- **Softmax:** Use exclusively in the output layer for mutually exclusive multi-class classification.
- **GELU:** Default choice for transformer-based architectures. Generally outperforms ReLU on NLP and vision transformer tasks.

3.2.4 Common Misconceptions

- **ReLU solves the vanishing gradient problem entirely.** ReLU avoids vanishing gradients for positive inputs, but neurons receiving consistently negative inputs produce zero gradients (the dying ReLU problem) which is its own form of learning failure.
- **Leaky ReLU is always better than ReLU.** The small negative slope does prevent dying neurons, but performance gains are not guaranteed and are often marginal. Empirical testing is always recommended.
- **Sigmoid and Softmax are interchangeable for classification.** Sigmoid is for *binary* classification (single output neuron). Softmax is for *multi-class* classification (one output per class). For multi-label problems where multiple classes can be correct simultaneously, independent Sigmoid outputs are preferred over Softmax.
- **A high Softmax output means the model is confident and correct.** Softmax outputs always sum to 1 regardless of input quality. On out-of-distribution inputs, the model can output high-confidence predictions that are entirely wrong.

- **Tanh is strictly better than Sigmoid for hidden layers.** Tanh is zero-centred and has stronger gradients, making it generally preferable in hidden layers. However, both activation functions still suffer from the vanishing gradient problem at saturation and have largely been displaced by ReLU-family functions in modern deep networks.
- **GELU is just a smooth version of ReLU.** While both gate inputs around zero, GELU uses a probabilistic mechanism based on the normal CDF, making it non-monotonic. This is a fundamentally different gating behaviour, not merely a smoothed threshold.

3.2.5 Key Terminology

Activation Function A transformation applied to a neuron’s pre-activation value to introduce non-linearity into the network.

Pre-activation The raw value passed into an activation function, typically a weighted sum of inputs plus a bias term.

Vanishing Gradient A training failure mode where gradients shrink exponentially as they propagate backwards through many layers, preventing early layers from learning.

Dying ReLU A failure mode specific to ReLU where a neuron consistently receives negative pre-activations, causing its gradient to become permanently zero and the neuron to stop learning.

Saturation The condition in which an activation function’s output is near its asymptotic limit, causing its gradient to approach zero.

Sparse Activation A property of ReLU whereby a large fraction of neurons output exactly zero for any given input, reducing computational load and encouraging efficient representations.

Logits The raw, unnormalised scores produced by the final linear layer of a network before an output activation such as Softmax is applied.

Gating The mechanism by which an activation function scales or suppresses its input, either sharply (as in ReLU) or smoothly (as in GELU).

3.3 Key Papers

1. *Gaussian Error Linear Units (GELUs)* [4].

URL: <https://arxiv.org/abs/1606.08415>

Summary: This paper introduces the Gaussian Error Linear Unit (GELU), an activation function defined as $x\Phi(x)$, where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution. Unlike ReLU, which gates inputs by their sign, GELU weights inputs by their magnitude relative to other inputs — a probabilistic interpretation derived from the expectation of a stochastic regulariser. The authors evaluate GELU against ReLU and ELU

across computer vision, NLP, and speech tasks including MNIST, CIFAR-10/100, TIMIT, and Twitter POS tagging, finding consistent performance improvements. Two practical approximations are also provided to reduce the computational cost of exact evaluation.

Relevance: This is the original paper proposing GELU. It directly justifies the probabilistic gating description and the two approximation formulas presented, and mentions that GELU subsequently became the default activation function in transformer architectures such as BERT and GPT.

3.4 Additional (Informal) Material

1. [YouTube] [A Review of 10 Most Popular Activation Functions in Neural Networks](#)
2. [YouTube] [Neural Networks From Scratch - Lec 15 - GeLU Activation Function](#)
3. [GeeksforGeeks] [Activation functions in Neural Networks](#)
4. [Medium] [Understanding Activation Functions in Neural Networks](#)
5. [Google] <https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions>

References

- [1] D. Scherer, A. Müller, and S. Behnke, “Evaluation of pooling operations in convolutional architectures for object recognition,” in *Artificial Neural Networks – ICANN 2010* (K. Diamantaras, W. Duch, and L. S. Iliadis, eds.), (Berlin, Heidelberg), pp. 92–101, Springer Berlin Heidelberg, 2010.
- [2] M. Lin, Q. Chen, and S. Yan, “Network in network,” 2014.
- [3] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” 2015.
- [4] D. Hendrycks and K. Gimpel, “Gaussian error linear units (gelus),” 2023.